

# Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

## Computer Algorithm Laboratory

### Assignment No. 6 — Dynamic Programming

**Name : Atharv V Kulkarni**

**PRN : 245109074**

**Class : TY B.Tech CSE (B)**

## Question 1: Robot Unique Paths with Obstacles

### Problem Statement:

A warehouse robot starts at the top-left corner of an  $m \times n$  grid and must reach the bottom-right corner, moving only right or down at each step. Some cells contain obstacles the robot cannot pass through. Find the number of distinct paths the robot can take to reach the package.

### Approach:

approach\_1: recursively try every right/down move from each cell and count paths reaching the end. tc:  $O(2^{m+n})$  sc:  $O(m+n)$

approach\_2 (optimal): bottom-up dp where each free cell's path count is the sum of the cell above and the cell to the left, zero at obstacles. tc:  $O(m \times n)$  sc:  $O(m \times n)$

**Time Complexity:**  $O(m \times n)$

**Space Complexity:**  $O(m \times n)$

### Code :

```
#include<iostream>
#include<vector>
using namespace std;
long long uniquePaths(vector<vector<int>>&grid){
    int m=grid.size(),n=grid[0].size();
    vector<vector<long long>>dp(m,vector<long long>(n,0));
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            if(grid[i][j]==1){
                dp[i][j]=0;
            }
        }
    }
}
```

```

        continue;
    }
    if(i==0&&j==0){
        dp[i][j]=1;
        continue;
    }
    long long fromtop=(i>0)?dp[i-1][j]:0;
    long long fromleft=(j>0)?dp[i][j-1]:0;
    dp[i][j]=fromtop+fromleft;
}
}
return dp[m-1][n-1];
}
int main(){
    vector<vector<int>>test1={{0,0,0},{0,1,0},{0,0,0}};
    vector<vector<int>>test2={{0,1},{0,0}};
    vector<vector<int>>test3={{1,0}};
    vector<vector<int>>test4={{0,0},{0,0}};
    vector<vector<int>>test5={{0,0,0,0},{0,1,0,0},{0,0,1,0},{0,0,0,0}};
    cout<<"test case 1: number of paths is "<<uniquePaths(test1)<<endl;
    cout<<"test case 2: number of paths is "<<uniquePaths(test2)<<endl;
    cout<<"test case 3: number of paths is "<<uniquePaths(test3)<<endl;
    cout<<"test case 4: number of paths is "<<uniquePaths(test4)<<endl;
    cout<<"test case 5: number of paths is "<<uniquePaths(test5)<<endl;
    return 0;
}

```

## Output:

```

Linux@atharvk % main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_6 at 07:33:04 PM
➤ mkdir -p build && g++ 1_robot_unique_paths.cpp -o build/1_robot_unique_paths && ./build/1_robot_unique_paths
test case 1: number of paths is 2
test case 2: number of paths is 1
test case 3: number of paths is 0
test case 4: number of paths is 2
test case 5: number of paths is 4

```

## Question 2: 0/1 Knapsack — Trek Packing

### Problem Statement:

You are packing a backpack with weight capacity  $W$  kg from  $n$  items, each with a weight and a usefulness value. Each item can either be taken completely or left behind. Maximize total usefulness value without exceeding capacity  $W$ .

### Approach:

approach\_1: try every subset of items and keep the best value among those within capacity. tc:  $O(2^n)$  sc:  $O(n)$

approach\_2 (optimal): bottom-up dp table over items and capacity, at each item choosing the better of skipping it or taking it if it fits. tc:  $O(n \times W)$  sc:  $O(n \times W)$

**Time Complexity:**  $O(n \times W)$

**Space Complexity:**  $O(n \times W)$

### Code :

```
#include<iostream>
#include<vector>
using namespace std;
int knapsack(vector<int>weights,vector<int>values,int capacity){
    int n=weights.size();
    vector<vector<int>>>dp(n+1,vector<int>(capacity+1,0));
    for(int i=1;i<=n;i++){
        for(int w=0;w<=capacity;w++){
            dp[i][w]=dp[i-1][w];
            if(weights[i-1]<=w){
                int taken=dp[i-1][w-weights[i-1]]+values[i-1];
                if(taken>dp[i][w])
                    dp[i][w]=taken;
            }
        }
    }
    return dp[n][capacity];
}
int main(){
    vector<int>w1={1,3,4,5};
    vector<int>v1={1,4,5,7};
    vector<int>w2={2,3,4,5};
    vector<int>v2={3,4,5,6};
    vector<int>w3={10,20,30};
    vector<int>v3={60,100,120};
    vector<int>w4={1,2,3};
    vector<int>v4={10,15,40};
    vector<int>w5={5};
    vector<int>v5={10};
    cout<<"test case 1: maximum value is "<<knapsack(w1,v1,7)<<endl;
```

```

cout<<"test case 2: maximum value is "<<knapsack(w2,v2,5)<<endl;
cout<<"test case 3: maximum value is "<<knapsack(w3,v3,50)<<endl;
cout<<"test case 4: maximum value is "<<knapsack(w4,v4,6)<<endl;
cout<<"test case 5: maximum value is "<<knapsack(w5,v5,3)<<endl;
return 0;
}

```

### Output:

```

Linux@atharvk ~ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assingment_6 at 07:33:36 PM
➤ mkdir -p build && g++ 2_knapsack_01.cpp -o build/2_knapsack_01 && ./build/2_knapsack_01
test case 1: maximum value is 9
test case 2: maximum value is 7
test case 3: maximum value is 220
test case 4: maximum value is 65
test case 5: maximum value is 0

```

## Question 3: Currency Kiosk — Minimum Coins and Number of Ways

### Problem Statement:

A currency kiosk wants to give change using the minimum number of notes/coins from a set of denominations available in unlimited supply. Part A: find the minimum number of coins needed to make a given amount, or -1 if not possible. Part B: find the total number of distinct ways to make that amount.

### Approach:

approach\_1: recursively try every denomination at each remaining amount and take the best/sum over all branches. tc:  $O(\text{denominations raised to amount})$  sc:  $O(\text{amount})$

approach\_2 (optimal): bottom-up dp over amounts, tracking the minimum coins for part a and the running combination count for part b. tc:  $O(\text{amount into number of denominations})$  sc:  $O(\text{amount})$

**Time Complexity:**  $O(\text{amount into number of denominations})$

**Space Complexity:**  $O(\text{amount})$

### Code :

```

#include<iostream>
#include<vector>
#include<climits>
using namespace std;
int minCoins(vector<int>coins,int amount){
    vector<int>dp(amount+1,INT_MAX);
    dp[0]=0;

```

```

    for(int i=1;i<=amount;i++){
        for(int c:coins){
            if(c<=i&&dp[i-c]!=INT_MAX){
                if(dp[i-c]+1<dp[i])
                    dp[i]=dp[i-c]+1;
            }
        }
    }
    return (dp[amount]==INT_MAX)?-1:dp[amount];
}

long long countWays(vector<int>coins,int amount){
    vector<long long>dp(amount+1,0);
    dp[0]=1;
    for(int c:coins){
        for(int i=c;i<=amount;i++){
            dp[i]+=dp[i-c];
        }
    }
    return dp[amount];
}

int main(){
    vector<int>c1={1,2,5};
    vector<int>c2={2};
    vector<int>c3={1,3,4};
    vector<int>c4={1,5,10,25};
    vector<int>c5={1,2,5};
    cout<<"test case 1: min coins "<<minCoins(c1,11)<<" ways "<<countWays(c1,11)<<endl;
    cout<<"test case 2: min coins "<<minCoins(c2,3)<<" ways "<<countWays(c2,3)<<endl;
    cout<<"test case 3: min coins "<<minCoins(c3,6)<<" ways "<<countWays(c3,6)<<endl;
    cout<<"test case 4: min coins "<<minCoins(c4,30)<<" ways "<<countWays(c4,30)<<endl;
    cout<<"test case 5: min coins "<<minCoins(c5,0)<<" ways "<<countWays(c5,0)<<endl;
    return 0;
}

```

## Output:

```

Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assingment_6 at 07:34:40 PM
➤ mkdir -p build && g++ 3_coin_change.cpp -o build/3_coin_change && ./build/3_coin_change
test case 1: min coins 3 ways 11
test case 2: min coins -1 ways 0
test case 3: min coins 2 ways 4
test case 4: min coins 2 ways 18
test case 5: min coins 0 ways 1

```

## Question 4: Longest Common Subsequence — Plagiarism Detection

### Problem Statement:

A plagiarism detection tool compares two essays to find how similar they are, based on the longest sequence of words appearing in the same relative order in both documents. Find the length of the Longest Common Subsequence between the two essays, and reconstruct the subsequence itself.

### Approach:

approach\_1: recursively branch on every pair of positions in both strings, matching or skipping one character at a time. tc:  $O(2^{n+m})$  sc:  $O(n+m)$

approach\_2 (optimal): bottom-up dp table over both strings' prefixes, extending the match on equal characters and taking the better of skipping either character otherwise, then backtracking through the table to rebuild the subsequence. tc:  $O(n \times m)$  sc:  $O(n \times m)$

**Time Complexity:**  $O(n \times m)$

**Space Complexity:**  $O(n \times m)$

### Code :

```
#include<iostream>
#include<vector>
using namespace std;
string lcs(string a,string b,int&length){
    int n=a.size(),m=b.size();
    vector<vector<int>>>dp(n+1,vector<int>(m+1,0));
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            if(a[i-1]==b[j-1])
                dp[i][j]=dp[i-1][j-1]+1;
            else
                dp[i][j]=(dp[i-1][j]>dp[i][j-1])?dp[i-1][j]:dp[i][j-1];
        }
    }
    length=dp[n][m];
    string result="";
    int i=n,j=m;
    while(i>0&&j>0){
        if(a[i-1]==b[j-1]){
            result=a[i-1]+result;
            i--;
            j--;
        }
        else if(dp[i-1][j]>dp[i][j-1])
            i--;
        else
```

```

        j--;
    }
    return result;
}
int main(){
    int length;
    string result;
    result=lcs("abcde","ace",length);
    cout<<"test case 1: lcs length "<<length<<" subsequence "<<result<<endl;
    result=lcs("abc","abc",length);
    cout<<"test case 2: lcs length "<<length<<" subsequence "<<result<<endl;
    result=lcs("abc","def",length);
    cout<<"test case 3: lcs length "<<length<<" subsequence "<<result<<endl;
    result=lcs("AGGTAB","GXTXAYB",length);
    cout<<"test case 4: lcs length "<<length<<" subsequence "<<result<<endl;
    result=lcs("machine","learning",length);
    cout<<"test case 5: lcs length "<<length<<" subsequence "<<result<<endl;
    return 0;
}

```

### Output:

```

❖Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_6 at 07:35:02 PM
❖ mkdir -p build && g++ 4_longest_common_subsequence.cpp -o build/4_longest_common_subsequence && ./build/4_longest_common_subsequence
test case 1: lcs length 3 subsequence ace
test case 2: lcs length 3 subsequence abc
test case 3: lcs length 0 subsequence
test case 4: lcs length 4 subsequence GTAB
test case 5: lcs length 3 subsequence ain

```

## Question 5: Minimum Edit Distance — DNA Sequence Comparison

### Problem Statement:

A biotech lab is comparing a mutated DNA sequence to a reference sequence. They want the minimum number of operations (insert, delete, substitute a base) required to convert the mutated sequence into the reference sequence. Compute the minimum edit distance between the two strings.

### Approach:

approach\_1: recursively try insert, delete and substitute at each mismatched position and take the cheapest branch. tc:  $O(3^{n+m})$  sc:  $O(n+m)$

approach\_2 (optimal): bottom-up dp table over both strings' prefixes, copying the diagonal value on a match and otherwise taking one plus the cheapest of insert, delete or substitute. tc:  $O(n \times m)$  sc:  $O(n \times m)$

**Time Complexity:**  $O(n \times m)$

**Space Complexity:**  $O(n \times m)$

### Code :

```
#include<iostream>
#include<vector>
using namespace std;
int editDistance(string a,string b){
    int n=a.size(),m=b.size();
    vector<vector<int>>>dp(n+1,vector<int>(m+1,0));
    for(int i=0;i<=n;i++){
        dp[i][0]=i;
    }
    for(int j=0;j<=m;j++){
        dp[0][j]=j;
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            if(a[i-1]==b[j-1])
                dp[i][j]=dp[i-1][j-1];
            else{
                int sub=dp[i-1][j-1];
                int del=dp[i-1][j];
                int ins=dp[i][j-1];
                int best=sub;
                if(del<best)
                    best=del;
                if(ins<best)
                    best=ins;
                dp[i][j]=1+best;
            }
        }
    }
}
```

```

    }
}
}
return dp[n][m];
}
int main(){
    cout<<"test case 1: edit distance is "<<editDistance("GATTACA","GCATGCU")<<endl;
    cout<<"test case 2: edit distance is "<<editDistance("kitten","sitting")<<endl;
    cout<<"test case 3: edit distance is "<<editDistance("abc","abc")<<endl;
    cout<<"test case 4: edit distance is "<<editDistance("", "abc")<<endl;
    cout<<"test case 5: edit distance is "<<editDistance("intention","execution")<<endl;
    return 0;
}

```

### Output:

```

❖Linux@atharvk ~ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assingment_6 at 07:35:39 PM
➤ mkdir -p build && g++ 5_edit_distance.cpp -o build/5_edit_distance && ./build/5_edit_distance
test case 1: edit distance is 4
test case 2: edit distance is 3
test case 3: edit distance is 0
test case 4: edit distance is 3
test case 5: edit distance is 5

```